

1. Ejercicio 11 – Automatización con Cucumber

Este ejercicio describe la automatización de los escenarios definidos en el Ejercicio 10, utilizando Cucumber como framework de testing BDD. La solución propuesta cumple con los requisitos de separación clara de responsabilidades, reutilización, parametrización real y justificación de decisiones de diseño.

1.1. Estructura del proyecto

La automatización se organiza con una separación explícita entre definición de comportamiento, implementación de pasos y utilidades de soporte.

```
project/
+-- features/
| +-- procesamiento_batch.feature
+-- step_definitions/
| +-- ejecucion_steps.java
| +-- archivos_steps.java
| +-- validaciones_steps.java
+-- support/
| +-- TestContext.java
| +-- FileHelper.java
| +-- CommandExecutor.java
+-- resources/
    +-- idiomas_validos.dat
    +-- palabras_validas.dat
    +-- palabras_error.dat
```

1.2. Separación de responsabilidades

- **Features:** contienen únicamente los escenarios en lenguaje Gherkin, describiendo el comportamiento esperado sin lógica técnica.
- **Step Definitions:** traducen los pasos Gherkin a acciones ejecutables, delegando la lógica compleja a componentes reutilizables.
- **Support:** encapsula la ejecución del programa COBOL, la preparación de archivos y el manejo de contexto compartido.

1.3. Parametrización real

Los escenarios permiten variar dinámicamente el parámetro de ejecución del programa, evitando valores hardcodeados.

```
@When("ejecuto el programa con el parametro {string}")
public void ejecuto_el_programa(String parametro) {
    CommandExecutor.run(parametro);
}
```

Esta estrategia permite reutilizar el mismo escenario con distintos valores de entrada, tal como lo exige la consigna.

1.4. Reutilización de pasos

Los pasos que describen la existencia y preparación de archivos son reutilizados en múltiples escenarios, evitando duplicación y facilitando el mantenimiento.

```
@Given("existe el archivo {string} con registros bien formados")
public void existe_archivo_valido(String nombreArchivo) {
    FileHelper.copyResource(nombreArchivo, "palabras.dat");
}
```

1.5. Manejo de contexto compartido

Se utiliza un contexto de prueba compartido para almacenar la salida estándar y el código de retorno del programa ejecutado.

```
public class TestContext {
    public static String stdout;
    public static int exitCode;

    public static void reset() {
        stdout = "";
        exitCode = -1;
    }
}
```

Este enfoque permite realizar múltiples validaciones sobre una misma ejecución sin acoplar los pasos entre sí.

1.6. Ejecución del programa COBOL real

La automatización ejecuta el binario real del programa COBOL, sin mocks, validando la integración completa entre archivos, parámetros y salida estándar.

```
public class CommandExecutor {

    public static void run(String parametro) {
        ProcessBuilder pb =
            new ProcessBuilder("./IBM_Cobol", parametro);

        Process process = pb.start();
        TestContext.stdout = readOutput(process);
        TestContext.exitCode = process.waitFor();
    }
}
```

1.7. Validaciones automatizadas

Las validaciones se realizan sobre la salida estándar y el estado de finalización del proceso.

```
@Then("se muestra un mensaje de error de formato invalido")
public void error_formato() {
    assertTrue(TestContext.stdout.contains("formato"));
}
```

1.8. Justificación de decisiones de diseño

- Se utiliza Cucumber para garantizar trazabilidad directa entre reglas, escenarios y código automatizado.
- Se prueba el binario COBOL real para evitar falsos positivos derivados de mocks o simulaciones.
- La separación en helpers reutilizables facilita la extensibilidad hacia nuevos escenarios sin reescritura de pasos existentes.

1.9. Escenarios no automatizados

No se automatizan pruebas de performance, concurrencia ni codificaciones regionales, ya que no forman parte del alcance definido en la consigna y no afectan el comportamiento funcional del sistema.